

ROS2

DAY 4 – Manual 2

ROS2 Services – Server & Client Implementation

Complete Beginner-Friendly

Objective

By the end of this manual, you will be able to:

1. Write a ROS2 **Service Server node**
 2. Write a ROS2 **Service Client node**
 3. Understand **each line of the code**
 4. Run and test a ROS2 service using string commands
-

Step 0 – Pre-Requisites

- Manual 1 completed (package created, `.srv` file ready, workspace built)
- ROS2 sourced:

```
source ~/ros2_ws/install/setup.bash
```

Step 1 – Create Service Server Node

1. Navigate to the package folder:

```
cd ~/ros2_ws/src/day4_services/day4_services
```

2. Create `string_server.py`:

```
nano string_server.py
```

3. Paste the code:

```
import rclpy
from rclpy.node import Node
from day4_services.srv import StringCommand
```

Explanation:

- `rclpy` → ROS2 Python client library
 - `Node` → base class to create ROS2 nodes
 - `StringCommand` → generated service interface from `.srv` file
-

4. Define the server class:

```
class StringServer(Node):
    def __init__(self):
        super().__init__('string_server')
        # Create a service named 'string_cmd' with type StringCommand
        self.srv = self.create_service(StringCommand, 'string_cmd',
self.callback)
        self.get_logger().info("String Command Server Ready")
```

Explanation:

- `Node('string_server')` → names this node
 - `create_service(...)` → registers a service with ROS2
 - `callback` → function called when client sends request
 - `get_logger().info()` → prints info to terminal
-

5. Create the callback function:

```
def callback(self, request, response):
    cmd = request.command
    self.get_logger().info(f"Received command: {cmd}")
    response.response = f"Server processed: {cmd}"
    return response
```

Explanation:

- `request.command` → reads data sent by client
 - `response.response` → prepares reply to client
 - `get_logger().info()` → shows the command received
-

6. Add the `main()` function:

```
def main():
    rclpy.init()
    node = StringServer()
    rclpy.spin(node)
```

```
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Explanation:

- `rclpy.init()` → initializes ROS2
 - `spin(node)` → keeps server running
 - `shutdown()` → clean exit when terminated
-

Step 2 – Create Service Client Node

1. In the same folder, create `string_client.py`:

```
nano string_client.py
```

2. Paste the code:

```
import rclpy
from rclpy.node import Node
from day4_services.srv import StringCommand
```

Explanation:

- Imports ROS2 libraries and service interface
-

3. Define the client class:

```
class StringClient(Node):
    def __init__(self):
        super().__init__('string_client')
        # Create client for 'string_cmd' service
        self.cli = self.create_client(StringCommand, 'string_cmd')
        # Wait until server is active
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().warn("Waiting for server...")
        self.req = StringCommand.Request()
```

Explanation:

- `create_client()` → connects to server service
- `wait_for_service()` → waits for server activation
- `StringCommand.Request()` → prepares request object

4. Send request function:

```
def send(self, text):
    self.req.command = text
    return self.cli.call_async(self.req)
```

Explanation:

- `req.command` → sets the string to send
 - `call_async()` → sends request and waits for server response
-

5. Add the `main()` function:

```
def main():
    rclpy.init()
    node = StringClient()
    # Send command to server
    future = node.send("Hello Server")
    # Wait for server response
    rclpy.spin_until_future_complete(node, future)
    print("Response:", future.result().response)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Explanation:

- Sends request "Hello Server"
 - Waits for server response
 - Prints server reply
-

Step 3 – Run the Service

Step 3.1 – Start Server

Open terminal:

```
ros2 run day4_services string_server
```

You will see:

```
[INFO] [string_server]: String Command Server Ready
```

Step 3.2 – Start Client

Open another terminal:

```
ros2 run day4_services string_client
```

You will see:

```
Response: Server processed: Hello Server
```

Server terminal shows:

```
[INFO] [string_server]: Received command: Hello Server
```

Step 3.3 – Test Multiple Commands

Modify client text:

```
future = node.send("Reset Motors")
```

Server responds:

```
Server processed: Reset Motors
```

Step 4 – Recap

- Services = **synchronous request/response**
 - Server waits for requests, client sends requests
 - `.srv` file defines the structure of request and response
 - Code explained **line by line** for beginners
 - Learners can now **build, run, and test ROS2 services**
-